

=====

CAPSTONE PROJECT

NOTEBOOK 42: FULL-161 SINGLE-FILE INFERENCE DEMO

=====

Purpose:

This notebook demonstrates the final full-161 inference

pipeline on a single audio file.

The goal is to:

1. Load the frozen Stage-1 expanded hybrid benchmark model
2. Load the Stage-2 rare-tail router
3. Accept either:
 - a known dataset test file, or
 - an external audio file

4. Extract structured descriptors for the structured branch

5. Extract a Mel spectrogram for the audio branch

6. Produce Stage-1 candidate-150 predictions

7. Produce Stage-2 rare-tail fallback suggestions

8. Save report-ready inference outputs

=====

In [1]:

```
# =====
# 1. IMPORT LIBRARIES
# =====

import os
import json
import warnings
from pathlib import Path

import joblib
import librosa
import numpy as np
import pandas as pd
import tensorflow as tf

from scipy.stats import skew, kurtosis

warnings.filterwarnings("ignore", category=FutureWarning)
warnings.filterwarnings("ignore", category=UserWarning)

SEED = 42
np.random.seed(SEED)
tf.random.set_seed(SEED)
```

```
print("Seed set to:", SEED)
print("TensorFlow version:", tf.__version__)
```

Seed set to: 42
TensorFlow version: 2.20.0

In [2]:

```
# =====
# 2. LOAD FROZEN ARTIFACTS
# =====

# ---- Structured branch artifacts
structured_model =
joblib.load("../models/final_structured_multilabel_candidate150_best_model.joblib")
structured_scaler =
joblib.load("../models/final_structured_multilabel_candidate150_scaler.joblib")

# ---- Audio branch artifact
audio_model =
tf.keras.models.load_model("../models/audio_multilabel_candidate150_expanded_final.
keras")

# ---- Candidate Label columns
candidate_label_cols = np.load(
    "../data/processed/hybrid_multilabel_candidate150_expanded_label_columns.npy",
    allow_pickle=True
)

# ---- Stage-1 benchmark config
stage1_config = {}
with
open("../data/processed/hybrid_multilabel_candidate150_expanded_best_config.txt",
"r") as f:
    for line in f:
        line = line.strip()
        if "=" in line:
            k, v = line.split("=")
            stage1_config[k.strip()] = float(v.strip())

STAGE1_STRUCTURED_WEIGHT = stage1_config["structured_weight"]
STAGE1_AUDIO_WEIGHT = stage1_config["audio_weight"]
STAGE1_THRESHOLD = stage1_config["threshold"]

# ---- Stage-2 router artifacts
rare_tail_router_df =
pd.read_csv("../data/processed/full161_rare_tail_routing_table.csv")
stage2_strategy_df =
pd.read_csv("../data/processed/full161_stage2_execution_strategy.csv")

stage2_config = {}
with open("../data/processed/full161_stage2_best_config.txt", "r") as f:
    for line in f:
        line = line.strip()
        if "=" in line:
            k, v = line.split("=")
```

```

        stage2_config[k.strip()] = v.strip()

STAGE2_ANCHOR_TRIGGER_THRESHOLD = float(stage2_config["anchor_trigger_threshold"])
STAGE2_TOP_K = int(stage2_config["top_k"])

# ---- Genre inventory and full master
genre_inventory_df = pd.read_csv("../data/processed/full_genre_inventory.csv")
full_master_df = pd.read_csv("../data/processed/multilabel_full_master_table.csv")

# ---- Reference structured feature columns
features_reference = pd.read_csv(
    "../data/raw/metadata/features.csv",
    header=[0, 1, 2],
    index_col=0
)

print("Structured model loaded.")
print("Audio model loaded.")
print("Candidate labels:", len(candidate_label_cols))
print("Stage-1 config:", stage1_config)
print("Stage-2 config:", stage2_config)
print("Rare-tail router shape:", rare_tail_router_df.shape)
print("Genre inventory shape:", genre_inventory_df.shape)
print("Full master shape:", full_master_df.shape)
print("Reference features shape:", features_reference.shape)

```

Structured model loaded.

Audio model loaded.

Candidate labels: 150

Stage-1 config: {'structured_weight': 0.1, 'audio_weight': 0.9, 'threshold': 0.2}

Stage-2 config: {'anchor_trigger_threshold': '0.1', 'top_k': '1'}

Rare-tail router shape: (13, 26)

Genre inventory shape: (163, 11)

Full master shape: (81574, 170)

Reference features shape: (106574, 518)

In [3]:

```

# =====
# 3. PREPARE LOOKUPS
# =====

genre_inventory_df["genre_id"] = genre_inventory_df["genre_id"].astype(int)

genre_name_map = dict(zip(genre_inventory_df["genre_id"],
                           genre_inventory_df["genre_name"]))

candidate_label_ids = [int(col.replace("genre_", "")) for col in
                       candidate_label_cols]
candidate_id_to_col = {int(col.replace("genre_", "")): col for col in
                       candidate_label_cols}
candidate_id_to_index = {int(col.replace("genre_", "")): i for i, col in
                          enumerate(candidate_label_cols)}

# Rare-tail labels that are actually routed
fallback_router_df = rare_tail_router_df[

```

```

rare_tail_router_df["fallback_mode"] == "Hierarchy-triggered fallback"
].copy().reset_index(drop=True)

fallback_router_df["rare_tail_genre_id"] =
fallback_router_df["rare_tail_genre_id"].astype(int)
fallback_router_df["anchor_candidate_id"] =
fallback_router_df["anchor_candidate_id"].astype(int)

inventory_only_df = rare_tail_router_df[
    rare_tail_router_df["fallback_mode"] == "Inventory only"
].copy().reset_index(drop=True)

# Flatten reference feature columns exactly like prior structured notebooks
features_reference.columns = [
    "_".join([str(level) for level in col]).strip()
    for col in features_reference.columns.to_flat_index()
]

reference_feature_df = features_reference.copy()
reference_feature_df = reference_feature_df.select_dtypes(include=["number"])
reference_feature_df = reference_feature_df.replace([np.inf, -np.inf], np.nan)
reference_feature_means = reference_feature_df.mean(axis=0)

reference_feature_columns = list(reference_feature_df.columns)

print("Fallback rare-tail labels:", fallback_router_df.shape[0])
print("Inventory-only rare-tail labels:", inventory_only_df.shape[0])
print("Structured reference columns:", len(reference_feature_columns))
display(fallback_router_df.head())
display(inventory_only_df.head())

```

Fallback rare-tail labels: 10
 Inventory-only rare-tail labels: 3
 Structured reference columns: 518

	rare_tail_genre_id	rare_tail_genre_name	parent_id	parent_name	root_genre_id	root_genre_
0	176	Pacific	2	International	2	Interna
1	1060	Tango	46	Latin America	2	Interna
2	465	Musical Theater	20	Spoken	20	Sp
3	189	Talk Radio	65	Radio	20	Sp
4	1032	Turkish	102	Middle East	2	Interna

5 rows × 26 columns

	rare_tail_genre_id	rare_tail_genre_name	parent_id	parent_name	root_genre_id	root_genre_
0	174	South Indian Traditional	86	Indian	2	Interna
1	175	Bollywood	86	Indian	2	Interna
2	178	Be-Bop	4	Jazz	4	

3 rows × 26 columns

In [4]:

```
# =====
# 4. SELECT INPUT MODE
# =====
# MODE OPTIONS:
# - "existing_test_track": use a track from the expanded test split
# - "external_file": use your own audio file path

MODE = "existing_test_track"

# If MODE == "existing_test_track", you can choose by row index
EXISTING_TEST_ROW_INDEX = 0

# If MODE == "external_file", set your local path here
EXTERNAL_AUDIO_PATH = "../data/raw/audio/fma_large/000/000183.mp3"

expanded_test_df =
pd.read_csv("../data/processed/audio_multilabel_candidate150_expanded_test.csv")
```

```

if MODE == "existing_test_track":
    selected_row = expanded_test_df.iloc[EXISTING_TEST_ROW_INDEX].copy()
    AUDIO_PATH = selected_row["audio_path"]
    TRACK_ID = int(selected_row["track_id"])
    INPUT_SOURCE = "existing_test_track"
else:
    AUDIO_PATH = EXTERNAL_AUDIO_PATH
    TRACK_ID = None
    INPUT_SOURCE = "external_file"

print("Input source:", INPUT_SOURCE)
print("Audio path:", AUDIO_PATH)
print("Track ID:", TRACK_ID)

```

Input source: existing_test_track
 Audio path: ../data/raw/audio/fma_large\000\000568.mp3
 Track ID: 568

In [5]:

```

# =====
# 5. AUDIO SETTINGS
# =====

SR = 22050
DURATION = 15
N_MELS = 64
N_FFT = 2048
HOP_LENGTH = 1024
MAX_FRAMES = int(np.ceil((DURATION * SR) / HOP_LENGTH)) + 1

print("SR:", SR)
print("DURATION:", DURATION)
print("N_MELS:", N_MELS)
print("MAX_FRAMES:", MAX_FRAMES)

```

SR: 22050
 DURATION: 15
 N_MELS: 64
 MAX_FRAMES: 324

In [6]:

```

# =====
# 6. HELPER FUNCTIONS
# =====

def load_audio(file_path, sr=SR, duration=DURATION):
    y, sr_loaded = librosa.load(file_path, sr=sr, mono=True, duration=duration)
    if y is None or len(y) == 0:
        raise ValueError(f"Could not load usable audio from: {file_path}")
    return y, sr_loaded

def build_mel_input(y, sr=SR, n_mels=N_MELS, n_fft=N_FFT, hop_length=HOP_LENGTH,
max_frames=MAX_FRAMES):

```

```

mel = librosa.feature.melspectrogram(
    y=y,
    sr=sr,
    n_fft=n_fft,
    hop_length=hop_length,
    n_mels=n_mels
)
mel_db = librosa.power_to_db(mel, ref=np.max)
mel_db = np.clip((mel_db + 80.0) / 80.0, 0.0, 1.0)

if mel_db.shape[1] < max_frames:
    pad_width = max_frames - mel_db.shape[1]
    mel_db = np.pad(mel_db, ((0, 0), (0, pad_width)), mode="constant")
else:
    mel_db = mel_db[:, :max_frames]

X_audio = mel_db.astype(np.float32)[None, :, :, None]
return X_audio

def safe_stat_vector(arr_2d, stat_name):
    """
    arr_2d shape: (n_components, n_frames)
    Returns a clean 1D float32 vector.
    """
    arr_2d = np.asarray(arr_2d, dtype=np.float64)

    if arr_2d.ndim == 1:
        arr_2d = arr_2d.reshape(1, -1)

    if stat_name == "mean":
        out = np.mean(arr_2d, axis=1)
    elif stat_name == "std":
        out = np.std(arr_2d, axis=1)
    elif stat_name == "median":
        out = np.median(arr_2d, axis=1)
    elif stat_name == "min":
        out = np.min(arr_2d, axis=1)
    elif stat_name == "max":
        out = np.max(arr_2d, axis=1)
    elif stat_name == "skew":
        out = skew(arr_2d, axis=1, bias=False, nan_policy="omit")
    elif stat_name == "kurtosis":
        out = kurtosis(arr_2d, axis=1, bias=False, nan_policy="omit")
    else:
        raise ValueError(f"Unknown stat: {stat_name}")

    out = np.asarray(out, dtype=np.float64)

    # Manual cleanup to avoid dtype casting issues
    out[~np.isfinite(out)] = 0.0

    return out.astype(np.float32)

def build_feature_matrices(y, sr=SR):
    y = np.asarray(y, dtype=np.float64)
    y_harmonic = librosa.effects.harmonic(y)

```



```

feature_mats = {}

feature_mats["chroma_stft"] = librosa.feature.chroma_stft(y=y, sr=sr)
feature_mats["chroma_cqt"] = librosa.feature.chroma_cqt(y=y, sr=sr)
feature_mats["chroma_cens"] = librosa.feature.chroma_cens(y=y, sr=sr)
feature_mats["tonnetz"] = librosa.feature.tonnetz(y=y_harmonic, sr=sr)
feature_mats["mfcc"] = librosa.feature.mfcc(y=y, sr=sr, n_mfcc=20)
feature_mats["rms"] = librosa.feature.rms(y=y)
feature_mats["spectral_centroid"] = librosa.feature.spectral_centroid(y=y,
sr=sr)
feature_mats["spectral_bandwidth"] = librosa.feature.spectral_bandwidth(y=y,
sr=sr)
feature_mats["spectral_contrast"] = librosa.feature.spectral_contrast(y=y,
sr=sr)
feature_mats["spectral_rolloff"] = librosa.feature.spectral_rolloff(y=y, sr=sr)
feature_mats["zcr"] = librosa.feature.zero_crossing_rate(y)

return feature_mats

def build_structured_feature_vector(y, reference_columns, reference_means, sr=SR):
    feature_mats = build_feature_matrices(y, sr=sr)

    row_dict = {}

    for col in reference_columns:
        # Example: chroma_cens_kurtosis_01
        parts = col.split("_")
        component_idx = int(parts[-1]) - 1
        stat_name = parts[-2]
        feature_name = "_".join(parts[:-2])

        if feature_name in feature_mats:
            mat = feature_mats[feature_name]
            stat_vec = safe_stat_vector(mat, stat_name)

            if 0 <= component_idx < len(stat_vec):
                row_dict[col] = float(stat_vec[component_idx])
            else:
                row_dict[col] = np.nan
        else:
            row_dict[col] = np.nan

    X_one = pd.DataFrame([row_dict], columns=reference_columns)
    X_one = X_one.replace([np.inf, -np.inf], np.nan)

    for col in reference_columns:
        if pd.isna(X_one.loc[0, col]):
            X_one.loc[0, col] = float(reference_means[col])

    X_one = X_one.astype(np.float32)
    return X_one

def scores_to_pseudopros(score_matrix):
    clipped = np.clip(score_matrix, -20, 20)
    return 1.0 / (1.0 + np.exp(-clipped))

def get_structured_scores(model, X_scaled):

```

```

if hasattr(model, "predict_proba"):
    scores = model.predict_proba(X_scaled)
elif hasattr(model, "decision_function"):
    scores = model.decision_function(X_scaled)
else:
    raise ValueError("Structured model supports neither predict_proba nor
decision_function.")
return np.asarray(scores)

def fuse_probabilities(structured_probs, audio_probs, w_structured, w_audio):
    return (w_structured * structured_probs) + (w_audio * audio_probs)

def decode_stage1(prob_matrix, threshold):
    return (prob_matrix >= threshold).astype(np.uint8)

def build_rare_tail_scores_for_single(stage1_probs, router_df,
candidate_index_map):
    scores = []

    for _, row in router_df.iterrows():
        anchor_id = int(row["anchor_candidate_id"])
        anchor_idx = candidate_index_map[anchor_id]

        anchor_prob = float(stage1_probs[0, anchor_idx])

        p_anchor = 0.0 if pd.isna(row["p_rare_given_anchor"]) else
float(row["p_rare_given_anchor"])
        p_root = 0.0 if pd.isna(row["p_rare_given_root"]) else
float(row["p_rare_given_root"])

        strength = max(p_anchor, p_root)
        rare_score = anchor_prob * strength

        scores.append({
            "rare_tail_genre_id": int(row["rare_tail_genre_id"]),
            "rare_tail_genre_name": row["rare_tail_genre_name"],
            "anchor_candidate_id": anchor_id,
            "anchor_candidate_name": row["anchor_candidate_name"],
            "anchor_prob": anchor_prob,
            "rare_tail_score": rare_score,
            "fallback_mode": row["fallback_mode"]
        })

    return pd.DataFrame(scores)

def get_ground_truth_for_existing_track(track_id, full_df, genre_map,
candidate_cols, fallback_router_df):
    row = full_df[full_df["track_id"] == track_id].iloc[0]

    true_candidate_ids = []
    for col in candidate_cols:
        if int(row[col]) == 1:
            true_candidate_ids.append(int(col.replace("genre_", "")))

    fallback_cols = [f"genre_{gid}" for gid in
fallback_router_df["rare_tail_genre_id"].astype(int).tolist()]
    true_rare_ids = []

```

```

for col in fallback_cols:
    if col in row.index and int(row[col]) == 1:
        true_rare_ids.append(int(col.replace("genre_", "")))

    true_candidate_names = [genre_map.get(gid, str(gid)) for gid in
true_candidate_ids]
    true_rare_names = [genre_map.get(gid, str(gid)) for gid in true_rare_ids]

    return true_candidate_ids, true_candidate_names, true_rare_ids, true_rare_names

```

In [7]:

```

# =====
# 7. LOAD AUDIO AND BUILD INPUTS
# =====

y, sr_loaded = load_audio(AUDIO_PATH, sr=SR, duration=DURATION)

X_audio_input = build_mel_input(
    y,
    sr=sr_loaded,
    n_mels=N_MELS,
    n_fft=N_FFT,
    hop_length=HOP_LENGTH,
    max_frames=MAX_FRAMES
)

X_structured_one = build_structured_feature_vector(
    y,
    reference_feature_columns,
    reference_feature_means,
    sr=sr_loaded
)

X_structured_scaled =
structured_scaler.transform(X_structured_one).astype(np.float32)

print("Loaded audio length (samples):", len(y))
print("Audio input shape:", X_audio_input.shape)
print("Structured feature vector shape:", X_structured_one.shape)
print("Structured scaled shape:", X_structured_scaled.shape)
display(X_structured_one.iloc[:, :20])

```

Loaded audio length (samples): 330750
Audio input shape: (1, 64, 324, 1)
Structured feature vector shape: (1, 518)
Structured scaled shape: (1, 518)

	chroma_cens_kurtosis_01	chroma_cens_kurtosis_02	chroma_cens_kurtosis_03	chroma_cens_ku
--	-------------------------	-------------------------	-------------------------	----------------

0	-0.440189	-0.503811	-0.651409	
---	-----------	-----------	-----------	--

In [8]:

```
# =====
# 8. STRUCTURED BRANCH PREDICTION
# =====

structured_scores = get_structured_scores(structured_model, X_structured_scaled)
structured_probs = scores_to_pseudopros(structured_scores)

print("Structured score shape:", structured_scores.shape)
print("Structured probability shape:", structured_probs.shape)
```

Structured score shape: (1, 150)

Structured probability shape: (1, 150)

In [9]:

```
# =====
# 9. AUDIO BRANCH PREDICTION
# =====

audio_probs = audio_model.predict(X_audio_input, verbose=0)

print("Audio probability shape:", audio_probs.shape)
```

Audio probability shape: (1, 150)

In [10]:

```
# =====
# 10. STAGE-1 HYBRID FUSION
# =====

stage1_probs = fuse_probabilities(
    structured_probs,
    audio_probs,
    STAGE1_STRUCTURED_WEIGHT,
    STAGE1_AUDIO_WEIGHT
)

stage1_pred = decode_stage1(stage1_probs, STAGE1_THRESHOLD)

print("Stage-1 fused probability shape:", stage1_probs.shape)
print("Stage-1 hard prediction shape:", stage1_pred.shape)
print("Number of Stage-1 predicted candidate labels:", int(stage1_pred.sum()))
```

Stage-1 fused probability shape: (1, 150)

Stage-1 hard prediction shape: (1, 150)

Number of Stage-1 predicted candidate labels: 4

In [11]:

```
# =====
# 11. BUILD STAGE-1 CANDIDATE OUTPUT TABLES
# =====

candidate_rows = []

for j, col in enumerate(candidate_label_cols):
    gid = int(col.replace("genre_", ""))
    candidate_rows.append({
        "genre_id": gid,
        "genre_name": genre_name_map.get(gid, str(gid)),
        "structured_probability": float(structured_probs[0, j]),
        "audio_probability": float(audio_probs[0, j]),
        "hybrid_probability": float(stage1_probs[0, j]),
        "predicted_stage1": int(stage1_pred[0, j])
    })

candidate_results_df = pd.DataFrame(candidate_rows).sort_values(
    ["predicted_stage1", "hybrid_probability"],
    ascending=[False, False]
).reset_index(drop=True)

predicted_candidate_df = candidate_results_df[
    candidate_results_df["predicted_stage1"] == 1
].copy().reset_index(drop=True)

top10_candidate_df = candidate_results_df.head(10).copy()

print("Predicted Stage-1 candidate labels:")
display(predicted_candidate_df)

print("Top 10 candidate labels by fused probability:")
display(top10_candidate_df)
```

Predicted Stage-1 candidate labels:

	genre_id	genre_name	structured_probability	audio_probability	hybrid_probability	predicted
0	12	Rock	2.061154e-09	0.665023	0.598521	
1	38	Experimental	2.498066e-01	0.302260	0.297014	
2	25	Punk	2.061154e-09	0.276578	0.248920	
3	15	Electronic	1.000000e+00	0.165195	0.248675	

◀  ▶

Top 10 candidate labels by fused probability:

	genre_id	genre_name	structured_probability	audio_probability	hybrid_probability	predict
0	12	Rock	2.061154e-09	0.665023	0.598521	
1	38	Experimental	2.498066e-01	0.302260	0.297014	
2	25	Punk	2.061154e-09	0.276578	0.248920	
3	15	Electronic	1.000000e+00	0.165195	0.248675	
4	10	Pop	2.061154e-09	0.197486	0.177738	
5	1235	Instrumental	1.000000e+00	0.055150	0.149635	
6	32	Noise	1.000000e+00	0.045236	0.140712	
7	107	Ambient	1.000000e+00	0.015106	0.113595	
8	85	Garage	2.061154e-09	0.125489	0.112940	
9	18	Soundtrack	9.673038e-01	0.016230	0.111337	

In [12]:

```

# =====
# 12. STAGE-2 RARE-TAIL FALLBACK SUGGESTIONS
# =====

rare_tail_scores_df = build_rare_tail_scores_for_single(
    stage1_probs,
    fallback_router_df,
    candidate_id_to_index
)

rare_tail_scores_df = rare_tail_scores_df.sort_values(
    ["rare_tail_score", "anchor_prob"],
    ascending=False
).reset_index(drop=True)

# Apply Stage-2 routing rule:
# only allow suggestions whose anchor candidate probability crosses the tuned
trigger
stage2_suggestions_df = rare_tail_scores_df[
    rare_tail_scores_df["anchor_prob"] >= STAGE2_ANCHOR_TRIGGER_THRESHOLD
].copy().reset_index(drop=True)

stage2_suggestions_df = stage2_suggestions_df.head(STAGE2_TOP_K).copy()

print("All Stage-2 rare-tail scores:")
display(rare_tail_scores_df)

print("Final Stage-2 rare-tail suggestions:")
display(stage2_suggestions_df)

```

All Stage-2 rare-tail scores:

	rare_tail_genre_id	rare_tail_genre_name	anchor_candidate_id	anchor_candidate_name	anchor
0	176	Pacific	2	International	0.
1	1032	Turkish	102	Middle East	0.
2	1060	Tango	46	Latin America	0.
3	374	Banter	20	Spoken	0.
4	465	Musical Theater	20	Spoken	0.
5	189	Talk Radio	65	Radio	0.
6	377	Deep Funk	19	Funk	0.
7	808	Salsa	46	Latin America	0.
8	173	N. Indian Traditional	86	Indian	0.
9	493	Western Swing	651	Country & Western	0.



Final Stage-2 rare-tail suggestions:

	rare_tail_genre_id	rare_tail_genre_name	anchor_candidate_id	anchor_candidate_name	anchor
--	--------------------	----------------------	---------------------	-----------------------	--------



In [13]:

```
# =====  
# 13. INVENTORY-ONLY LABELS  
# =====  
  
inventory_only_labels_df = inventory_only_df[  
    ["rare_tail_genre_id", "rare_tail_genre_name", "anchor_candidate_name",  
    "root_candidate_name", "fallback_mode"]  
].copy()
```

```
print("Inventory-only rare-tail labels (not directly surfaced as predictions):")
display(inventory_only_labels_df)
```

Inventory-only rare-tail labels (not directly surfaced as predictions):

	rare_tail_genre_id	rare_tail_genre_name	anchor_candidate_name	root_candidate_name	fallback_candidate_name
0	174	South Indian Traditional	Indian	International	Inventory-only
1	175	Bollywood	Indian	International	Inventory-only
2	178	Be-Bop	Jazz	Jazz	Inventory-only

In [14]:

```
# =====
# 14. OPTIONAL GROUND TRUTH FOR EXISTING DATASET TRACK
# =====

ground_truth_summary = {}

if INPUT_SOURCE == "existing_test_track" and TRACK_ID is not None:
    true_candidate_ids, true_candidate_names, true_rare_ids, true_rare_names =
get_ground_truth_for_existing_track(
    TRACK_ID,
    full_master_df,
    genre_name_map,
    candidate_label_cols,
    fallback_router_df
)

    ground_truth_summary = {
        "track_id": TRACK_ID,
        "true_candidate_ids": true_candidate_ids,
        "true_candidate_names": true_candidate_names,
        "true_rare_tail_ids": true_rare_ids,
        "true_rare_tail_names": true_rare_names
    }

    print("Ground truth for selected existing test track:")
    print(json.dumps(ground_truth_summary, indent=2))
else:
    print("No ground truth shown because MODE is external_file.")
```


Ground truth for selected existing test track:

```
{
  "track_id": 568,
  "true_candidate_ids": [
    12
  ],
  "true_candidate_names": [
    "Rock"
  ],
  "true_rare_tail_ids": [],
  "true_rare_tail_names": []
}
```

In [15]:

```
# =====
# 15. BUILD FINAL SINGLE-FILE SUMMARY
# =====

stage1_candidate_ids = predicted_candidate_df["genre_id"].astype(int).tolist()
stage1_candidate_names = predicted_candidate_df["genre_name"].tolist()

stage2_rare_tail_ids =
stage2_suggestions_df["rare_tail_genre_id"].astype(int).tolist() if
len(stage2_suggestions_df) > 0 else []
stage2_rare_tail_names = stage2_suggestions_df["rare_tail_genre_name"].tolist() if
len(stage2_suggestions_df) > 0 else []
stage2_rare_tail_scores =
stage2_suggestions_df["rare_tail_score"].round(6).tolist() if
len(stage2_suggestions_df) > 0 else []

final_summary = {
  "input_source": INPUT_SOURCE,
  "audio_path": AUDIO_PATH,
  "track_id": TRACK_ID,
  "stage1_config": {
    "structured_weight": STAGE1_STRUCTURED_WEIGHT,
    "audio_weight": STAGE1_AUDIO_WEIGHT,
    "threshold": STAGE1_THRESHOLD
  },
  "stage2_config": {
    "anchor_trigger_threshold": STAGE2_ANCHOR_TRIGGER_THRESHOLD,
    "top_k": STAGE2_TOP_K
  },
  "stage1_candidate_label_count": len(stage1_candidate_ids),
  "stage1_candidate_label_ids": stage1_candidate_ids,
  "stage1_candidate_label_names": stage1_candidate_names,
  "stage2_rare_tail_suggestion_count": len(stage2_rare_tail_ids),
  "stage2_rare_tail_suggestion_ids": stage2_rare_tail_ids,
  "stage2_rare_tail_suggestion_names": stage2_rare_tail_names,
  "stage2_rare_tail_suggestion_scores": stage2_rare_tail_scores,
  "inventory_only_rare_tail_labels":
inventory_only_labels_df["rare_tail_genre_name"].tolist(),
}

if ground_truth_summary:
  final_summary["ground_truth"] = ground_truth_summary
```

```
print("Final single-file inference summary:")
print(json.dumps(final_summary, indent=2))
```

Final single-file inference summary:

```
{
  "input_source": "existing_test_track",
  "audio_path": "../data/raw/audio/fma_large\\000\\000568.mp3",
  "track_id": 568,
  "stage1_config": {
    "structured_weight": 0.1,
    "audio_weight": 0.9,
    "threshold": 0.2
  },
  "stage2_config": {
    "anchor_trigger_threshold": 0.1,
    "top_k": 1
  },
  "stage1_candidate_label_count": 4,
  "stage1_candidate_label_ids": [
    12,
    38,
    25,
    15
  ],
  "stage1_candidate_label_names": [
    "Rock",
    "Experimental",
    "Punk",
    "Electronic"
  ],
  "stage2_rare_tail_suggestion_count": 0,
  "stage2_rare_tail_suggestion_ids": [],
  "stage2_rare_tail_suggestion_names": [],
  "stage2_rare_tail_suggestion_scores": [],
  "inventory_only_rare_tail_labels": [
    "South Indian Traditional",
    "Bollywood",
    "Be-Bop"
  ],
  "ground_truth": {
    "track_id": 568,
    "true_candidate_ids": [
      12
    ],
    "true_candidate_names": [
      "Rock"
    ],
    "true_rare_tail_ids": [],
    "true_rare_tail_names": []
  }
}
```

In [16]:

```

# =====
# 16. SAVE OUTPUTS
# =====

os.makedirs("../data/processed", exist_ok=True)

candidate_results_df.to_csv(
    "../data/processed/full161_single_file_candidate_results.csv",
    index=False
)

predicted_candidate_df.to_csv(
    "../data/processed/full161_single_file_stage1_predictions.csv",
    index=False
)

stage2_suggestions_df.to_csv(
    "../data/processed/full161_single_file_stage2_suggestions.csv",
    index=False
)

inventory_only_labels_df.to_csv(
    "../data/processed/full161_single_file_inventory_only_labels.csv",
    index=False
)

summary_df = pd.DataFrame([
    "input_source": final_summary["input_source"],
    "audio_path": final_summary["audio_path"],
    "track_id": final_summary["track_id"],
    "stage1_candidate_label_count": final_summary["stage1_candidate_label_count"],
    "stage2_rare_tail_suggestion_count":
final_summary["stage2_rare_tail_suggestion_count"]
])

summary_df.to_csv(
    "../data/processed/full161_single_file_inference_summary.csv",
    index=False
)

with open("../data/processed/full161_single_file_inference_summary.json", "w") as f:
    json.dump(final_summary, f, indent=2)

print("Saved full-161 single-file inference outputs.")

```

Saved full-161 single-file inference outputs.

In [17]:

```

# =====
# 17. INTERPRETATION NOTES
# =====

print("1. This notebook demonstrates the frozen full-161 inference pipeline on one

```

```
audio file.")
print("2. Stage 1 produces direct candidate-150 labels using the expanded hybrid
benchmark model.")
print("3. Stage 2 produces rare-tail fallback suggestions using the hierarchy-
triggered router.")
print("4. Inventory-only rare-tail labels are retained as taxonomy metadata and are
not directly predicted.")
print("5. This notebook can be used as the final deployment-style demo for the
capstone.")
```

1. This notebook demonstrates the frozen full-161 inference pipeline on one audio file.
2. Stage 1 produces direct candidate-150 labels using the expanded hybrid benchmark model.
3. Stage 2 produces rare-tail fallback suggestions using the hierarchy-triggered router.
4. Inventory-only rare-tail labels are retained as taxonomy metadata and are not directly predicted.
5. This notebook can be used as the final deployment-style demo for the capstone.